

AI WeatherWise API – Project Documentation

Project Title: AI WeatherWise API

Team Lead:

Sushvamitha.Y

Email: sushmasaran672@gmail.com

Team Members:

Faikha Tabasum.S

Email: s.faikhathabasum@gmail.com

Bairavi. S

Email: bairavis765@gmail.com

Jeevitha. M

Email: mjeevitha0105@gmail.com

PROJECT DESCRIPTION

AI WeatherWise API is a RESTful backend application developed using Node.js and Express.js that provides weather information, user authentication, favorite location management, and AI-powered weather insights. The system uses MongoDB with Mongoose to store user and location data. It integrates weather data with AI-generated summaries and practical recommendations, providing users with simple and useful information about weather conditions. The project follows a modular backend architecture using routes, controllers, models, middleware, and services.

Objectives

- Provide RESTful APIs for weather retrieval.
- Implement secure registration, login and profile retrieval.
- Protect private APIs using JSON Web Tokens (JWT).
- Store user accounts and favorite locations in MongoDB.
- Allow users to add, view, update and delete favorite locations.
- Generate concise AI weather summaries and practical recommendations.
- Maintain application functionality through deterministic fallback logic when external API keys are unavailable.
- Provide a Postman collection for API testing.

System Overview

The application follows an MVC-oriented layered architecture. Incoming HTTP requests are received by Express routes, validated and processed by controllers, and then passed to models or service modules as required. Authentication middleware verifies JWT tokens before private operations. External weather and AI services are accessed from dedicated service modules.

Technologies Used

- Node.js – JavaScript runtime for the backend.
- Express.js – Web framework for REST API routing and middleware.
- MongoDB – NoSQL database for users and favorite locations.
- Mongoose – ODM used to define schemas and communicate with MongoDB.
- JWT / jsonwebtoken – Token-based authentication and authorization.
- bcryptjs – Password hashing and password comparison.
- Google Gemini – AI generation of weather summaries and recommendations.
- OpenWeatherMap – External source for current weather information.
- dotenv – Loads environment variables from .env.
- CORS – Cross-origin resource sharing middleware.
- Postman – API testing using the included collection.

Project Structure

Backend/

```
|— src/
|   |— config/
|   |   └─ db.js
|   |— controllers/
|   |   |— authController.js
|   |   |— locationController.js
|   |   |— weatherController.js
|   |   └─ aiController.js
|   |— middleware/
|   |   └─ authMiddleware.js
|   |— models/
|   |   └─ User.js
```

```
| | └── Location.js
| | └── routes/
| |   ├── authRoutes.js
| |   ├── locationRoutes.js
| |   ├── weatherRoutes.js
| |   └── aiRoutes.js
| └── services/
|   ├── weatherService.js
|   └── aiService.js
└── app.js
    └── server.js
└── postman_collection.json
└── package.json
└── package-lock.json
└── .env
└── README.md
```

Explanation of Folders

config

Contains configuration modules, including the MongoDB connection.

controllers

Contains functions that validate requests, invoke services/models, and return API responses.

middleware

Contains authentication middleware used to protect private routes.

models

Defines MongoDB schemas using Mongoose.

routes

Maps HTTP methods and URL paths to controller functions.

services

Handles OpenWeatherMap and Gemini AI integration and fallback logic.

app.js

Configures Express, middleware, routes and error handling.

server.js

Main entry point that establishes the database connection and starts the server.

Features

- User registration and login
- JWT-based authentication
- User profile retrieval
- Add favorite location
- View favorite locations
- Update favorite location
- Delete favorite location
- Current weather retrieval
- AI weather summary
- AI weather recommendation
- REST API based architecture
- MongoDB database integration
- Deterministic fallback weather and AI logic

API Operations

Register User

Creates a new user account and returns a JWT.

Login User

Authenticates a user and returns a JWT.

Get Profile

Returns the authenticated user's profile.

Create Favorite Location

Adds a favorite city/location for the authenticated user.

Read Favorite Locations

Retrieves the user's stored favorite locations.

Update Favorite Location

Modifies an existing favorite location owned by the user.

Delete Favorite Location

Removes a favorite location owned by the user.

Get Weather

Retrieves current weather information for a city.

Weather Summary

Generates a concise AI-based weather summary.

Weather Recommendation

Generates practical recommendations based on weather conditions.

API Endpoints

- POST /api/auth/register – Create a new user account.
- POST /api/auth/login – Authenticate a user.
- GET /api/auth/profile – Get the authenticated user's profile.
- POST /api/locations – Add a favorite location.
- GET /api/locations – List favorite locations.
- PUT /api/locations/:id – Update a favorite location.
- DELETE /api/locations/:id – Delete a favorite location.
- GET /api/weather/:city – Retrieve current weather.
- POST /api/ai/weather-summary – Generate an AI weather summary.
- POST /api/ai/weather-recommendation – Generate an AI weather recommendation.

Database Design

User Collection

- name – Required string.
- email – Required, unique, lowercase and validated.
- password – Required string with a minimum of 6 characters and stored as a bcrypt hash.
- createdAt / updatedAt – Automatically managed by Mongoose timestamps.

Location Collection

- city – Required string.
- country – Required string.
- user – Required ObjectId reference to User.
- createdAt / updatedAt – Automatically managed by Mongoose timestamps.

A compound unique index is defined on user + city + country to prevent duplicate favorite locations for the same user.

Security Features

- Passwords are hashed with bcryptjs before database storage.
- JWT tokens are used for authenticated API access.

- Protected routes verify token validity before processing user-specific data.
- Favorite-location operations verify that the authenticated user owns the requested record.
- User profile retrieval excludes the password field.
- Environment variables are used for database credentials, JWT secrets and external API keys.

External Service Integration

OpenWeatherMap

The weather service calls the OpenWeatherMap current-weather endpoint with the city name, API key and metric units. The application exposes the required weather fields such as city, temperature, humidity, wind speed and condition.

Google Gemini

The AI service uses the Google Generative AI SDK and Gemini 2.5 Flash. Prompts request short, natural and practical output. If Gemini is unavailable or an API call fails, the service uses local rule-based generation.

Fallback and Resilience

- Weather requests can return deterministic mock values when the OpenWeatherMap key is absent or invalid.
- AI endpoints can generate local summaries and recommendations without Gemini.
- Deterministic fallback logic makes local development and API testing possible even before external credentials are configured.

How to Run the Project

1. Install Node.js 18 or later and npm.
2. Open a terminal in the Backend project directory.
3. Run `npm install` to install dependencies.
4. Start development mode with `npm run dev`.
5. Use `npm start` for normal server execution.

API Testing with Postman

The project includes `postman_collection.json` with sample requests for authentication, favorite locations, weather and AI insights. The collection uses variables for `base_url`, `token` and `location_id`. Private requests use Bearer token authentication.

6. Register a user.

7. Log in and obtain the JWT token.
8. Request the user profile.
9. Add a favorite location and record its ID.
10. List, update and delete favorite locations.
11. Request weather for a city.
12. Call the AI weather summary endpoint.
13. Call the AI weather recommendation endpoint.

Error Handling

Controllers use try/catch blocks and return structured JSON error messages. Express also defines a 404 handler for unknown routes and a centralized error handler for unhandled exceptions. Database startup failures prevent the server from running.

- 400 – Invalid or incomplete request data.
- 401 – Authentication/authorization failure.
- 404 – Requested city or resource was not found.
- 500 – Unexpected server-side failure.

Non-Functional Characteristics

- Security – JWT authentication and bcrypt password hashing.
- Maintainability – Separation into routes, controllers, services, models and middleware.
- Reliability – External-service fallback mechanisms.
- Testability – Postman collection and deterministic mock weather.
- Scalability foundation – Stateless JWT authentication and modular service architecture.
- Data integrity – Mongoose validation and unique location index.

Current Limitations and Possible Enhancements

- Add request-rate limiting and stronger API abuse protection.
- Use stricter production validation and sanitization for all user inputs.
- Add structured logging and monitoring.
- Add automated unit and integration tests.
- Move production secrets to a secure secrets-management system.
- Add refresh-token or session-management strategy if required by the frontend.
- Add pagination if favorite-location collections become large.
- Add weather forecasts and historical weather data.

- Add richer AI personalization using user preferences or saved locations.
- Deploy the backend behind HTTPS with production-grade configuration.

Advantages

- Modular and maintainable backend architecture.
- Secure authentication and user-specific data handling.
- Efficient data management using MongoDB.
- RESTful API design.
- AI-assisted interpretation of weather information.
- Fallback logic for development and testing.
- Easy integration with frontend applications.

Conclusion

AI WeatherWise API provides a modular backend for a weather-focused application with authentication, persistent favorite locations, live weather retrieval, and AI-assisted interpretation. Its MVC-style organization, JWT security, MongoDB data layer, external-service integrations and fallback mechanisms make the implementation suitable for frontend integration and academic project demonstration. The included Postman collection supports straightforward API verification.